

iPhone application development: Auto Layout, Views, Outlets and Actions, Different View Controller: single view Controller, Master Detail View Controller, Navigation View Controller, Managing Application Memory, Application delegate, Handling Keyboard Input

1. Auto Layout

Auto Layout dynamically adjusts your UI to fit screen sizes and orientations.

Example: Center a Button in a View

- A button is placed at the center of the screen, maintaining its position in all orientations.

Storyboard:

1. Drag a UIButton onto the canvas.
2. Set constraints:
 - Horizontally center (`Align Center X to: Superview`).
 - Vertically center (`Align Center Y to: Superview`).

Code Implementation:

```
```swift
let button = UIButton(type: .system)
button.setTitle("Click Me", for: .normal)
button.translatesAutoresizingMaskIntoConstraints = false
view.addSubview(button)

// Add Constraints
NSLayoutConstraint.activate([
 button.centerXAnchor.constraint(equalTo: view.centerXAnchor),
 button.centerYAnchor.constraint(equalTo: view.centerYAnchor),
 button.heightAnchor.constraint(equalToConstant: 50),
 button.widthAnchor.constraint(equalToConstant: 150)
])
```

---
```

2. Views

Views are building blocks of an app's UI.

Example: Adding a Label and a Button

A label displays a message; clicking the button changes the label's text.

Storyboard:

1. Drag a UILabel and UIButton onto the canvas.
2. Create `IBOutlet` for the label and `IBAction` for the button.

Code Implementation:

```
```swift
@IBOutlet weak var myLabel: UILabel!
```

```
@IBAction func buttonTapped(_ sender: UIButton) {
 myLabel.text = "Button Clicked!"
}
...
```

## Outlets and Actions:

### Outlets

An outlet is a variable that allows your code to reference and manipulate UI elements (like buttons, labels, or text fields) defined in Interface Builder.

### Steps to Create an Outlet:

1. Open Interface Builder:
  - Open your storyboard or XIB file in Xcode.
2. Show the Assistant Editor:
  - Open the Assistant Editor (two overlapping circles icon) to display your storyboard and corresponding view controller code side by side.
3. Control-Drag to Code:
  - Select the UI element in the storyboard.
  - Hold the Control key, click the element, and drag it to the desired location in your code file (usually inside the class definition of your view controller).
4. Define the Outlet:
  - When you release the drag, a popup appears. Name your outlet, ensure the type matches the UI element, and confirm the connection by clicking Connect.

### Example:

```
```swift
class ViewController: UIViewController {
    @IBOutlet weak var myLabel: UILabel!
}
...
```
```

### Actions

An action is a method triggered when a user interacts with a UI element, such as tapping a button.

### Steps to Create an Action:

1. Open Interface Builder:
  - Open your storyboard or XIB file in Xcode.
2. Show the Assistant Editor:
  - Open the Assistant Editor to display your storyboard and corresponding view controller code side by side.
3. Control-Drag to Code:
  - Select the UI element (e.g., a button).
  - Hold the Control key, click the element, and drag it to your code file.
4. Define the Action:
  - When you release the drag, a popup appears. Choose Action as the connection type, name the action method, and confirm the connection by clicking Connect.

Example:

```
```swift
class ViewController: UIViewController {
    @IBAction func buttonTapped(_ sender: UIButton) {
        print("Button was tapped!")
    }
}
```

Storyboard:

- Add a button and link its `Touch Up Inside` event to `changeBackgroundColor`.

4. Different View Controllers

Single View Controller:

-Default starting template in Xcode.

Master-Detail View Controller:

-Implemented using `UISplitViewController` for iPads or hierarchical navigation for iPhones.

Navigation View Controller:

1. Embed a View Controller in a `UINavigationController` (Editor -> Embed In -> Navigation Controller).

2. Push to a new View Controller:

```
```swift
let secondVC = SecondViewController()
navigationController?.pushViewController(secondVC, animated: true)
```
```

5. Managing Application Memory

As far as iPhone is concerned, it uses two main ways to store data, 1. the disk, 2. Random Access Memory. However, when an app runs on an iPhone device, a file that contains all the executable instructions is loaded into the RAM. The app also needs some memory to maintain the heap since all the objects of our classes reside in it.

Memory management is the process of managing this heap memory. It includes the life cycle management of the heap objects so that the objects get freed when they are no longer needed to rescue the memory.

However, in swift, the reference types are allocated on the heap memory whereas, the value types are not allocated on the heap. It is an important task to maintain the heap memory because our objects are large enough to contain a heavy part of the memory if they don't get freed. The app might get into a crash if it keeps running low on the memory.

Automatic Reference Counting (ARC). It automatically frees up the memory used by the class instances when we don't require them. However, as we have already stated in this article, Automatic Reference Counting (ARC) applies only to the class instances. Structure and Enumerations are Value types in swift, and therefore, their instances are not counted.

ARC allocates the chunk of memory to store information about the instance whenever created in our app code. The information contains the instance's type, including the value of any stored property associated with the instance.

ARC also frees up the memory when the instance is no longer needed. In this scenario, the memory used by that instance gets freed and used for other purposes instead. This also ensures that the instances don't take up unnecessary space in the memory when they are not used.

Concept: Use ARC (Automatic Reference Counting) to manage memory automatically.

Example: Preventing Strong Reference Cycles

```
```swift
class MyViewController: UIViewController {
 var closure: (() -> Void)?

 override func viewDidLoad() {
 super.viewDidLoad()

 closure = { [weak self] in
 self?.view.backgroundColor = .red
 }
 }
}
```
```

6. Application Delegate

Example: Logging App States

```
```swift
@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate {
 func applicationDidEnterBackground(_ application: UIApplication) {
 print("App Entered Background")
 }

 func applicationWillTerminate(_ application: UIApplication) {
 print("App Will Terminate")
 }
}
```
```

7. Handling Keyboard Input

Example: Dismissing the Keyboard on Tap

```
```swift
override func viewDidLoad() {
 super.viewDidLoad()

 let tap = UITapGestureRecognizer(target: self, action: #selector(dismissKeyboard))
 view.addGestureRecognizer(tap)
}

@objc func dismissKeyboard() {
 view.endEditing(true)
}
```
```

Adjusting View When Keyboard Appears:

```
```swift
NotificationCenter.default.addObserver(self,
 selector: #selector(keyboardWillShow),
 name: UIResponder.keyboardWillShowNotification,
 object: nil)

@objc func keyboardWillShow(notification: Notification) {
 if let keyboardFrame = notification.userInfo?[UIResponder.keyboardFrameEndUserInfoKey] as?
CGRect {
 view.frame.origin.y = -keyboardFrame.height
 }
}
```

---
```